OrangeHRM Test Automation

AI-Powered Test Case Generation & Automation Framework

Node.js Google Gemini AI Playwright Java + Maven ExcelJS

Project: OrangeHRM Login Page Test Automation

Target URL: https://opensource-demo.orangehrmlive.com/web/index.php/auth/login

Prepared by: K Jyothi Prakash

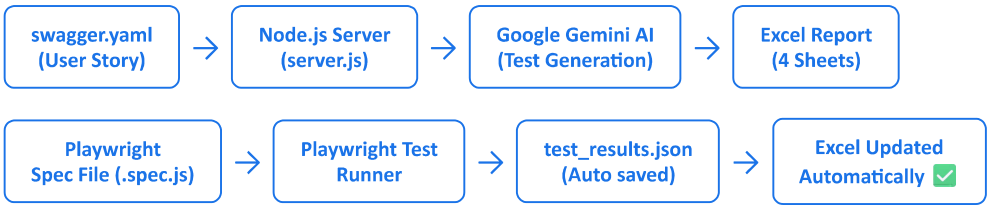
Date: 2025

1. Project Overview

This project is an **AI-powered end-to-end test automation framework** built for the OrangeHRM login page. It automatically generates manual test cases, creates Playwright automation scripts, executes them on a real browser, and updates an Excel report with the results — all with a single click using a batch file.

The system uses **Google Gemini AI** to intelligently generate test scenarios and test cases based on a user story defined in a Swagger YAML file. The generated test cases cover positive, negative, security, boundary, and UI validation scenarios.

2. Architecture & Flow



3. How to Run (1 Click)

The entire project can be executed with a single double-click on the batch file:

```
run-tests.bat
```

This batch file automatically opens 4 terminals and executes all steps in sequence.

4. Step-by-Step Execution

Step 1 — Start Node.js Server

```
node server.js
```

Starts the backend server on **http://localhost:3000**. This is the core of the project — it connects to Google Gemini AI, generates test cases, creates Excel reports, and updates results after automation runs.

Step 2 — Serve swagger.yaml

```
npx serve -p 5050
```

Serves the **swagger.yaml** file on **http://localhost:5050**. The swagger file contains the user story for the OrangeHRM login page which is used by Gemini AI to generate relevant test cases.

Step 3 — Generate Test Cases

```
mvn -q test -Dtest=GenerateFromSwaggerTest
```

Runs the Java test using Maven which calls the Node server API. The server then:

- Reads the user story from swagger.yaml
- Scrapes OrangeHRM login page fields using Cheerio
- Sends story + page fields to Google Gemini AI
- Gemini generates Test Scenarios and Test Cases
- Saves them to an Excel file with 4 sheets
- Generates a Playwright automation spec file

Step 4 — Run Playwright Automation Tests

```
npx playwright test --headed
```

Opens a real browser and runs all generated test cases automatically. After all tests finish, **global-teardown.js** runs automatically and updates the Excel file with results.

5. Test Cases Covered

#	Test Case	Type

1	Valid login with correct credentials	Positive
2	Invalid username	Negative
3	Invalid password	Negative
4	Both fields empty	Negative
5	Username empty, password filled	Negative
6	Username filled, password empty	Negative
7	SQL injection in username	Security
8	SQL injection in password	Security
9	XSS/HTML injection in username	Security
10	XSS/HTML injection in password	Security
11	Case sensitivity for username	Boundary
12	Case sensitivity for password	Boundary
13	Very long username (256 chars)	Boundary
14	Very long password (256 chars)	Boundary
15	Forgot password link presence	UI Validation
16	Password field masking	UI Validation

6. Excel Report Structure

Sheet 1 — Test Scenario

Contains high-level test scenarios grouped by module.

Module	Scenario ID	Scenario Name	Scenario Description	Requirement ID
OrangeHRM Login	SCN001	Valid Login	Verify login with valid credentials	REQ001

Sheet 2 — Test Cases

Contains detailed test cases. **Actual Result, Pass/Fail, Defect ID, Remarks** are filled automatically after automation run.

Test Scenario ID	Test Case ID	Description	Prerequisites	Steps	Expected Results	Actual Result	Pass/Fail	Defect ID	Remarks
SCN001	TC001	Valid login	Browser open	1. Enter Admin...	Dashboard shown	✅ Auto filled	✅ Auto filled	✅ Auto filled	✅ Auto filled

Sheet 3 — Defect Report

Automatically populated with defects for **failed test cases only** after automation run.

Serial No.	Defect ID	Description	Reproducible	Steps to Reproduce	Severity	Priority	Reported By	Reported Date	Status	Remarks
1	DEF001	✅ Auto filled	Yes	Manual	Medium	Medium	Manual	✅ Auto filled	Open	Manual

Sheet 4 — RTM (Requirements Traceability Matrix)

Links requirements to test scenarios, test cases and defects. **Defect ID** is filled automatically.

Serial No.	Requirement ID	Requirement Description	Test Scenario ID	Test Case ID	Defect ID
1	REQ001	Login functionality	SCN001	TC001	✅ Auto filled

7. Folder Structure

```
swagger\
  |-- Automation-test-Cases\    <-- Generated Playwright spec files
  |-- output\                   <-- Generated Excel files & test_results.json
  |-- .env                      <-- GEMINI_API_KEY (secret)
  |-- server.js                 <-- Main Node.js backend server
  |-- swagger.yaml              <-- API definition + user story
  |-- playwright.config.js      <-- Playwright configuration
  |-- global-teardown.js        <-- Auto updates Excel after tests
  |-- run-tests.bat             <-- 1 click runner
  |-- package.json              <-- Node.js dependencies

ai-testcase-gen\
  |-- src\test\java\com\jyothi\
  |    GenerateFromSwaggerTest.java  <-- Java test that calls Node server API
  |-- pom.xml                   <-- Maven dependencies
```

8. Technologies Used

Technology	Purpose
Node.js + Express	Backend server — handles API requests and orchestrates the flow
Google Gemini AI	Generates test scenarios, test cases and Playwright automation code
Playwright	Runs automation tests on real browser

ExcelJS	Creates and updates Excel reports with 4 sheets
Java + Maven	Triggers the test case generation API call
RestAssured	Java HTTP client to call the Node.js server
Cheerio	Scrapes OrangeHRM login page HTML to extract form fields
swagger.yaml	Provides API definition and user story for AI test generation
dotenv	Manages secret API keys securely via .env file

Node.js Express Google Gemini AI Playwright ExcelJS Java Maven RestAssured Cheerio YAML JavaScript (ESM)

9. API Endpoints

Method	Endpoint	Description
POST	/generate-testcases-from-swagger	Generates test cases + Excel + Playwright spec file
POST	/update-excel-results	Updates Excel with Playwright test results